

Лабораторная работа №4

Вычисление наибольшего общего делителя

11 Марта 2026 - Ли Хан

2026

Российский университет дружбы народов, Москва, Россия

Цель работы

я хочу представить результаты моей четвертой лабораторной работы. Моей целью было освоение принципов различных алгоритмов вычисления наибольшего общего делителя. Я стремился не просто получить результат, а через программную практику сравнить логические различия между классическим алгоритмом Евклида и бинарными методами, оптимизированными для компьютера, а также изучить линейное представление НОД через расширенные алгоритмы.

Детальный разбор четырех алгоритмов

Посмотрите на мою функцию `euclidean_gcd`. Этот алгоритм основан на теореме о делении с остатком

- Функционал кода: Сначала я инициализирую `r0` и `r1`. В цикле `while r1 != 0` программа выполняет основную операцию взятия остатка `r0 % r1`.
- Процесс реализации: На каждой итерации я присваиваю делитель переменной `r0`, а остаток — переменной `r1`.
- Получение результата: Когда остаток `r1` становится равным 0, цикл останавливается. В `r0` сохраняется последний ненулевой остаток, который и является $HON(a, b)$.

```
def euclidean_gcd(a, b):  
    # 中文: 初始化余数 r0 为 a, r1 为 b  
    # RU: Инициализация r0 = a, r1  
    r0, r1 = a, b  
  
    # 中文: 循环执行取余操作, 直到余数 r1 变为 0  
    # RU: Цикл продолжается, по  
    while r1 != 0:  
        # 中文: 核心逻辑:  $r_{i+1} = r_{i-1} \bmod r_i$   
        # RU: Основная логика:  
        r0, r1 = r1, r0 % r1  
  
    # 中文: 返回最后的非零余数作为最大公约数  
    # RU: Возвращает последний  
    return r0
```

```
# 样例运行 / Пример запуска
# 输入 (12345, 24690) -> 输出: 12345
print(f"НОД (12345, 24690) = {euclidean_gcd(12345, 24690)}")

НОД (12345, 24690) = 12345
```

Рис. 2: Алгоритм Евклида результат

Алгоритм 2: Бинарный алгоритм Евклида

Далее идет функция `binary_gcd`. Поскольку компьютеры обрабатывают двоичные сдвиги быстрее деления, этот алгоритм избегает трудоемких операций.

- Функционал кода: Я использую переменную `g` для извлечения общего множителя 2 из обоих чисел.
- Процесс реализации: В основном цикле я использую логику `while u % 2 == 0`, чтобы убрать все четные множители из `u` и `v`. Когда оба числа становятся нечетными, я выполняю вычитание `u - v` или `v - u`
- Получение результата: Этот процесс уменьшает масштаб чисел до тех пор, пока `u` не станет равным 0. В конце я умножаю извлеченный множитель `g` на `v`, получая итоговый результат.

Бинарный алгоритм Евклида компилятора

```
def binary_gcd(a, b):
    # 中文: g 用于记录提取出的公因子 2
    # RU: g используется для хранения
    g = 1

    # 中文: 当 a 和 b 都是偶数时, 不断提取公因子 2
    # RU: Пока a и b четные, выносим
    while a % 2 == 0 and b % 2 == 0:
        a //= 2
        b //= 2
        g *= 2

    u, v = a, b
    # 中文: 主循环, 通过移位和减法代替取模运算
    # RU: Основной цикл: сдвиги и вы
    while u != 0:
        while u % 2 == 0: u //= 2
        while v % 2 == 0: v //= 2
        if u >= v:
            u = u - v
        else:
            v = v - u

    # 中文: 结果为提取出的 2 的幂次与剩余值的乘积
    # RU: Результат — произведение
    return g * v
```

```
# 样例运行 / Пример запуска
# 输入 (48, 18) -> 输出: 6
print(f"Бинарный НОД (48, 18) = {binary_gcd(48, 18)}")

Бинарный НОД (48, 18) = 6
```

Рис. 4: Бинарный алгоритм Евклида результат

Теперь обратите внимание на функцию `extended_euclidean_gcd`. Она находит не только НОД, но и коэффициенты x и y для уравнения $ax + by = d$.

- Функционал кода: В коде я дополнительно содержу последовательности коэффициентов x и y
- Процесс реализации: На каждом шаге деления с остатком я получаю частное q и обновляю не только остаток, но и коэффициенты по формуле $x_{i-1} - q \cdot x_i$.
- Получение результата: Когда остаток r_{i+1} равен 0, функция возвращает НОД d вместе с коэффициентами x и y .

```
def extended_euclidean_gcd(a, b):  
    # 中文: 初始化系数 x 和 y, 满足  $ax + by = r$   
    # RU: Инициализация коэффициентов  
    r0, r1 = a, b  
    x0, x1 = 1, 0  
    y0, y1 = 0, 1  
  
    while r1 != 0:  
        # 中文: 计算商 q 用于后续系数更新  
        # RU: Находим частное q д:  
        q = r0 // r1  
        r0, r1 = r1, r0 - q * r1  
        # 中文: 更新线性组合的系数  
        # RU: Обновление коэффициентов  
        x0, x1 = x1, x0 - q * x1  
        y0, y1 = y1, y0 - q * y1  
  
    # 中文: 返回最大公约数 d 以及系数 x, y  
    # RU: Возвращает НОД d и коэф  
    return r0, x0, y0
```

```
# 样例运行 / Пример запуска
# 输入 (91, 105) -> 输出: (7, 7, -6) -> 即  $91*7 + 105*(-6) = 7$ 
d, x, y = extended_euclidean_gcd(91, 105)
print(f"Расширенный НОД: d={d}, x={x}, y={y}")
```

Расширенный НОД: d=7, x=7, y=-6

Рис. 6: Расширенный алгоритм Евклида результат

Алгоритм 4: Расширенный бинарный алгоритм Евклида

Это самая сложная функция `extended_binary_gcd`. Она находит d , x , y вообще без использования операции деления.

- Функционал кода: При обработке четного u я не просто делю его на 2, но и одновременно корректирую коэффициенты A и B , проверяя их четность
- Процесс реализации: Если коэффициенты нечетные, код выполняет операцию типа $(A + b)/2$, чтобы вычисления оставались в рамках целых чисел.
- Получение результата: Этот метод сочетает эффективность бинарного алгоритма и функциональность расширенного, выдавая на выходе d , x , y .

Расширенный бинарный алгоритм Евклида компилятора

```
def extended_binary_gcd(a, b):
    g = 1
    while a % 2 == 0 and b % 2 == 0:
        a //= 2; b //= 2; g *= 2

    u, v = a, b
    # 中文: A, B, C, D 对应线性组合的辅助变量
    # RU: A, B, C, D — вспомогательные переменные
    A, B, C, D = 1, 0, 0, 1

    while u != 0:
        while u % 2 == 0:
            u //= 2
            # 中文: 若系数为偶数则除以 2, 否则按公式调整
            # RU: Если коэффициенты четные
            if A % 2 == 0 and B % 2 == 0:
                A //= 2; B //= 2
            else:
                A = (A + b) // 2; B = (B - a) // 2
        # ... 对 v 执行类似逻辑 ...
        while v % 2 == 0:
            v //= 2
            if C % 2 == 0 and D % 2 == 0:
                C //= 2; D //= 2
            else:
                C = (C + b) // 2; D = (D - a) // 2

        if u >= v:
            u -= v; A -= C; B -= D
        else:
            v -= u; C -= A; D -= B

    return g * v, C, D
```

```
# 样例运行 / Пример запуска
# 输入 (154, 105) -> 输出: (7, 1, -1) -> 即  $154*1 + 105*(-1) = 49$  (示
d, x, y = extended_binary_gcd(154, 105)
print(f"Бинарный расширенный НОД: d={d}, x={x}, y={y}")

Бинарный расширенный НОД: d=7, x=13, y=-19
```

Рис. 8: Расширенный бинарный алгоритм Евклида результат

Итоги работы

В ходе этой работы я успешно реализовал все изученные алгоритмы поиска НОД. Я обнаружил, что хотя классический алгоритм Евклида наиболее прост логически, бинарный алгоритм демонстрирует более высокую эффективность на низком уровне за счет сдвигов и вычитания. Линейное представление, полученное с помощью расширенных алгоритмов, заложило математическую основу для моего дальнейшего изучения модульной инверсии и сравнений. Спасибо за внимание!